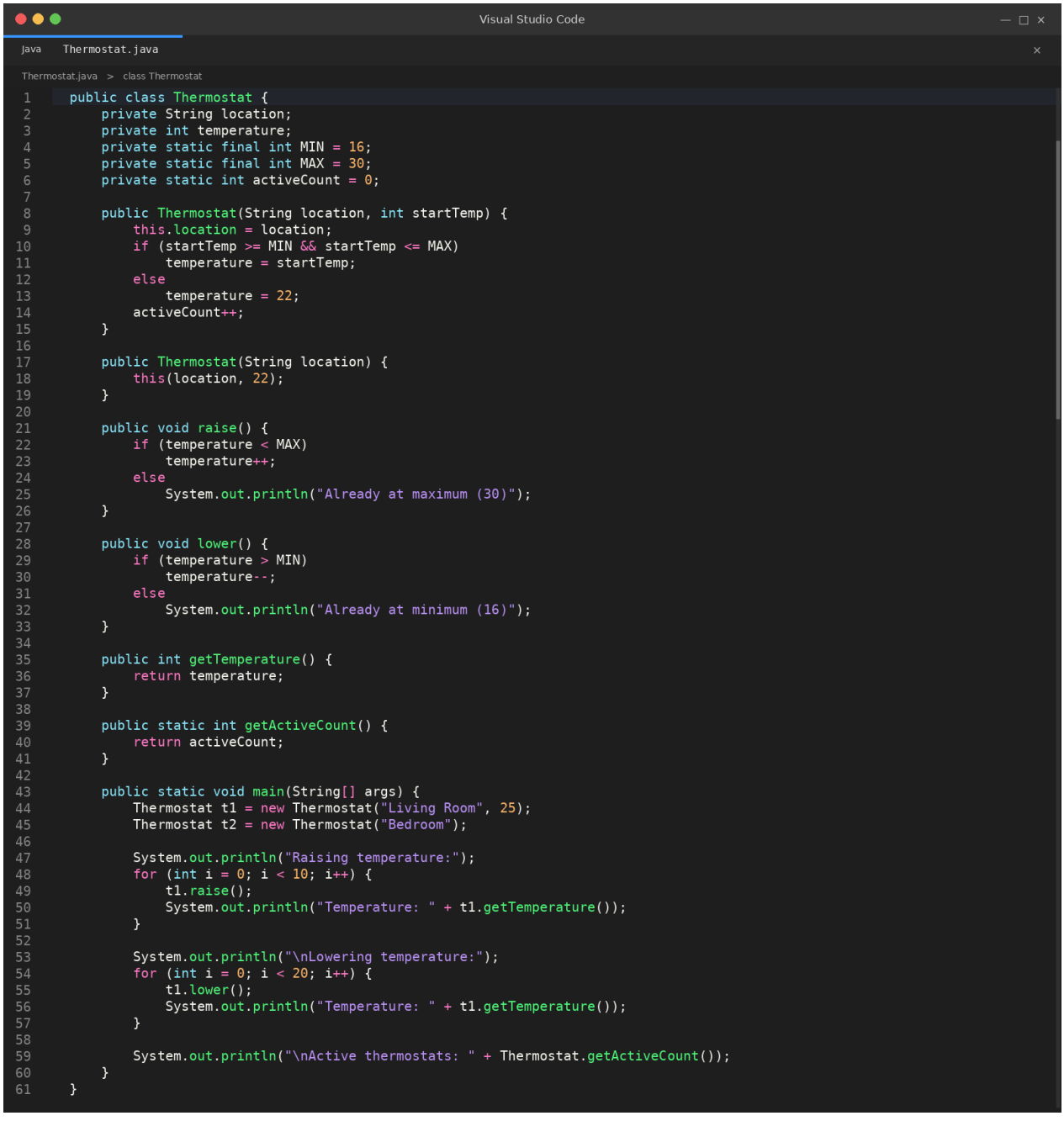
PRACTICAL - 02

Aim: Smart thermostat (Thermostat.java)

Code Screenshot:




```java
public class Thermostat {
    private String location;
    private int temperature;
    private static final int MIN = 16;
    private static final int MAX = 30;
    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;
        if (startTemp >= MIN && startTemp <= MAX)
            temperature = startTemp;
        else
            temperature = 22;
        activeCount++;
    }

    public Thermostat(String location) {
        this(location, 22);
    }

    public void raise() {
        if (temperature < MAX)
            temperature++;
        else
            System.out.println("Already at maximum (30)");
    }

    public void lower() {
        if (temperature > MIN)
            temperature--;
        else
            System.out.println("Already at minimum (16)");
    }

    public int getTemperature() {
        return temperature;
    }

    public static int getActiveCount() {
        return activeCount;
    }

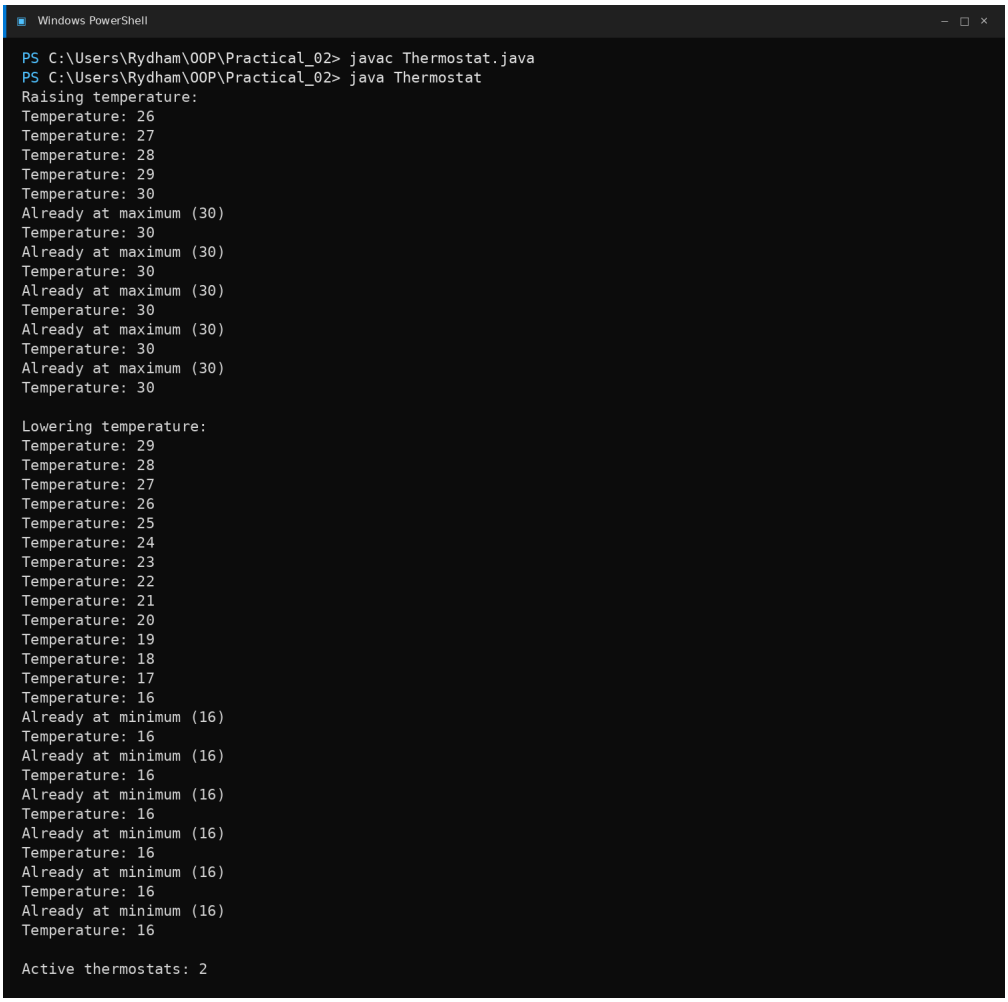
    public static void main(String[] args) {
        Thermostat t1 = new Thermostat("Living Room", 25);
        Thermostat t2 = new Thermostat("Bedroom");

        System.out.println("Raising temperature:");
        for (int i = 0; i < 10; i++) {
            t1.raise();
            System.out.println("Temperature: " + t1.getTemperature());
        }

        System.out.println("\nLowering temperature:");
        for (int i = 0; i < 20; i++) {
            t1.lower();
            System.out.println("Temperature: " + t1.getTemperature());
        }

        System.out.println("\nActive thermostats: " + Thermostat.getActiveCount());
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned encapsulation principles by restricting direct field access with private.
- Implemented constructor overloading and constructor chaining using this(...).
- Managed shared class state across instances using static variables.
- Enforced boundary constraints within mutator methods.
- Understood how objects maintain independent internal state.

Aim: Cinema show (CinemaShow.java)

Code Screenshot:

Visual Studio Code

Java CinemaShow.java

CinemaShow.java > class CinemaShow

```java
public class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        seatsAvailable = capacity;
    }

    public CinemaShow(String title) {
        this(title, 100);
    }

    public boolean book(int n) {
        if (n > 0 && n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

    public void cancel(int n) {
        if (n > 0) {
            seatsAvailable += n;
            if (seatsAvailable > capacity)
                seatsAvailable = capacity;
        }
    }

    public int getSeatsAvailable() {
        return seatsAvailable;
    }

    public static int getTotalBooked() {
        return totalBooked;
    }

    public static void main(String[] args) {
        CinemaShow show = new CinemaShow("Evening Show", 5);
        CinemaShow matinee = new CinemaShow("Matinee");

        System.out.println("Matinee seats: " + matinee.getSeatsAvailable());

        System.out.println("Book 3: " + show.book(3));
        System.out.println("Seats available: " + show.getSeatsAvailable());

        System.out.println("Book 2: " + show.book(2));
        System.out.println("Seats available: " + show.getSeatsAvailable());

        System.out.println("Book 1: " + show.book(1));
        System.out.println("Seats available: " + show.getSeatsAvailable());

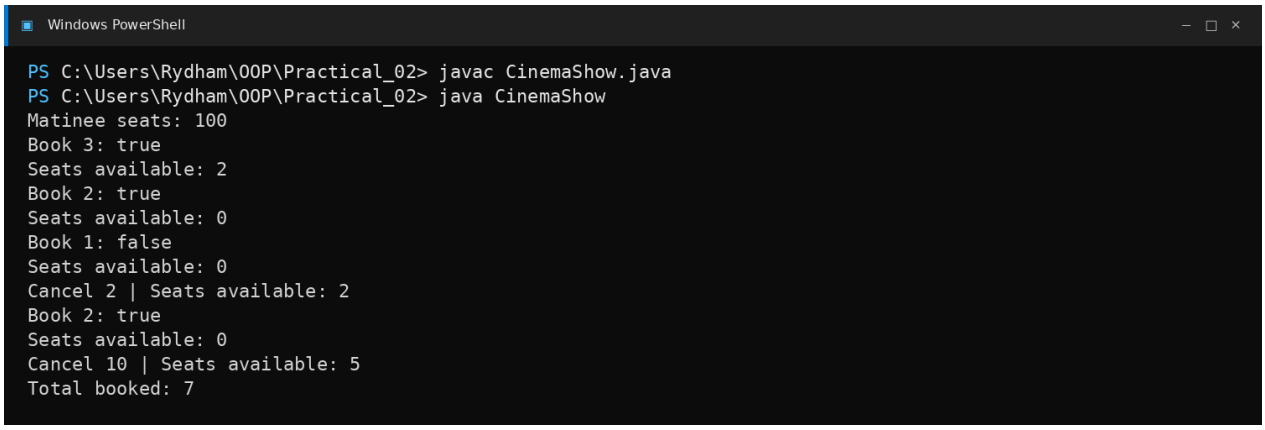
        show.cancel(2);
        System.out.println("Cancel 2 | Seats available: " + show.getSeatsAvailable());

        System.out.println("Book 2: " + show.book(2));
        System.out.println("Seats available: " + show.getSeatsAvailable());

        show.cancel(10);
        System.out.println("Cancel 10 | Seats available: " + show.getSeatsAvailable());

        System.out.println("Total booked: " + CinemaShow.getTotalBooked());
    }
}
```

Output Screenshot:

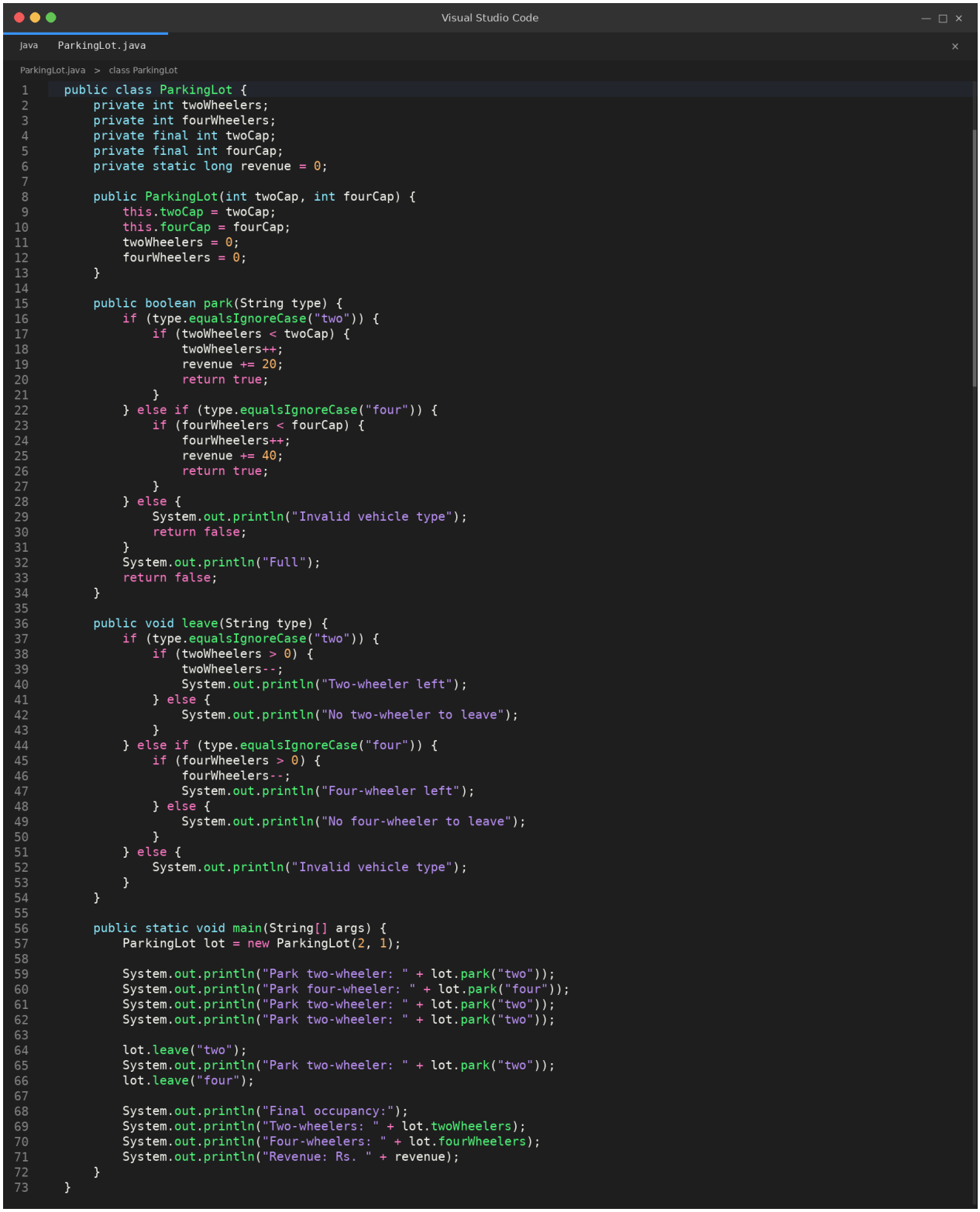


Learning Outcome:

- Mastered modeling real-world transactional entities in Java.
- Implemented defensive programming logic to protect capacity bounds.
- Tracked global operational metrics with static member variables.
- Applied boolean response status for state-modifying actions.
- Practiced clean method signatures and separation of concerns.

Aim: Parking lot (ParkingLot.java) - Advanced Learners

Code Screenshot:




```java
public class ParkingLot {
    private int twoWheelers;
    private int fourWheelers;
    private final int twoCap;
    private final int fourCap;
    private static long revenue = 0;

    public ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
        twoWheelers = 0;
        fourWheelers = 0;
    }

    public boolean park(String type) {
        if (type.equalsIgnoreCase("two")) {
            if (twoWheelers < twoCap) {
                twoWheelers++;
                revenue += 20;
                return true;
            }
        } else if (type.equalsIgnoreCase("four")) {
            if (fourWheelers < fourCap) {
                fourWheelers++;
                revenue += 40;
                return true;
            }
        } else {
            System.out.println("Invalid vehicle type");
            return false;
        }
        System.out.println("Full");
        return false;
    }

    public void leave(String type) {
        if (type.equalsIgnoreCase("two")) {
            if (twoWheelers > 0) {
                twoWheelers--;
                System.out.println("Two-wheeler left");
            } else {
                System.out.println("No two-wheeler to leave");
            }
        } else if (type.equalsIgnoreCase("four")) {
            if (fourWheelers > 0) {
                fourWheelers--;
                System.out.println("Four-wheeler left");
            } else {
                System.out.println("No four-wheeler to leave");
            }
        } else {
            System.out.println("Invalid vehicle type");
        }
    }

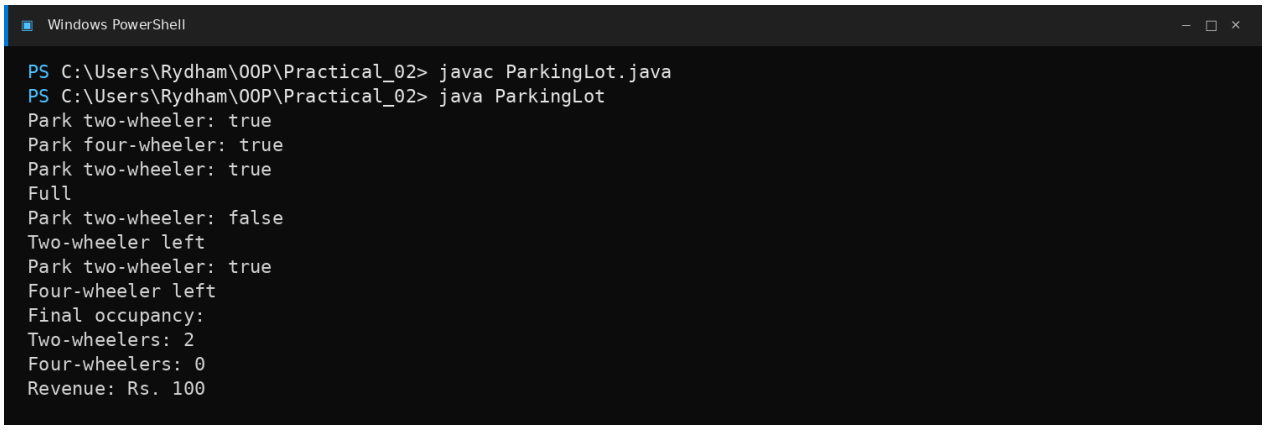
    public static void main(String[] args) {
        ParkingLot lot = new ParkingLot(2, 1);

        System.out.println("Park two-wheeler: " + lot.park("two"));
        System.out.println("Park four-wheeler: " + lot.park("four"));
        System.out.println("Park two-wheeler: " + lot.park("two"));
        System.out.println("Park two-wheeler: " + lot.park("two"));

        lot.leave("two");
        System.out.println("Park two-wheeler: " + lot.park("two"));
        lot.leave("four");

        System.out.println("Final occupancy:");
        System.out.println("Two-wheelers: " + lot.twoWheelers);
        System.out.println("Four-wheelers: " + lot.fourWheelers);
        System.out.println("Revenue: Rs. " + revenue);
    }
}
```

Output Screenshot:



Learning Outcome:

- Applied encapsulation to manage parking occupancy and capacity.
- Used final fields to enforce fixed parking-section limits.
- Implemented conditional logic to accept or reject vehicles.
- Updated static revenue based on successful parking events.
- Maintained valid occupancy when vehicles enter and leave.